Discussion 7 User Mode Kernel Mode and I/O Operations

Discussion Topic:

In your initial discussion post:

- Give at least two (2) reasons that modern operating systems switch from user to kernel mode to perform I/O operations, such as turning on a webcam or using a microphone.
- Then, provide three (3) other examples of such I/O operations. For each example, describe the process for doing this in general terms.
- Are the reasons that you stated worth the overhead of switching from user mode to kernel mode, and then back to user mode?
- If you were to design an OS, what would you do differently?

My Post:

Hello Class

In Operating Systems (OSs), users or applications directly accessing hardware is too risky and complicated; therefore, OSs implement a user mode and a kernel mode to perform I/O operations using a kernel.
Applications usually run in user mode, while drivers and devices run in kernel mode, allowing them to access hardware resources (Microsoft, 2025). For example, an application, in user mode, that requires access to a webcam or microphone is not allowed to directly control these devices' registers, memory buffers, interrupts, or drivers. Instead, the application has to request access through the OS, which switches to kernel mode to control, through drivers, the I/O operations, acting as an abstract layer between the application and the hardware.

The main reason that OSs switch between user mode for applications and kernel mode for drivers is to create an abstraction layer that protects and secures private data. I/O devices, such as cameras or microphones, can expose private video data, audio data, data files, network traffic, or keyboard input. For example, if user programs could directly control devices, a malfunctioning, corrupt, or malicious program could capture, modify, or delete data, as well as overwrite memory or interfere with another process. Another reason is for coordination and stability. This kernel abstraction layer can act as a managing layer for I/O operations. This is important because I/O devices operate at widely different speeds, use different data formats, and transfer methods. In other words, OSs through their kernel must handle many types of readable data, machine-readable data, and communication devices, while still providing efficient and uniform I/O operations (Colorado State University Global, n.d.).

An example of I/O operation is writing/reading files on disk or USB storage. SSDs or USB flash drives are used by users and applications to store and access files; to read and write files. For example, after a use/application requests a file be saved, the OS enters kernel mode, first it checks the users/applications the file permissions, locates the file system structures, places data into a kernel buffer or disk cache, and sends the request to the storage drive; then the device driver may use DMA to move data from memory to the device. When the storing process is complete, the device driver messages the OS through an interrupt or completion signal, and the OS will notify the user/application if any error occurred.

Another example I/O operation is network communications. For example, when an application needs to send data over Wi-Fi or Ethernet, it usually writes to a network socket for communication with another device, server, or online service. Then the OS enters kernel mode, and it prepares the data within the socket as network packets, initiates routing and protocol rules, and finally passes the packets to the network device driver. Once the network adapter gets the packet, it transmits it. Similarly, receiving data follows the reverse path; the adapter receives packets -> the driver -> the OS kernel network system processes them -> the OS delivers the data to the application.

Printing is another example of I/O operations. For example, when a user prints a document using an application, the application does not directly control the printer hardware. Instead, it sends the print job to the OS print system. The OS checks the job, selects the printer, communicates with the printer driver, and sends the data through USB, Wi-Fi, or a network print protocol.

The overhead of switching from user mode to kernel mode and back, in my opinion, is worth it. The switching process does cost time, but the cost is more than justified by security, stability, and device coordination benefits. Additionally, within the switching process, OSs can reduce overhead by using buffering, asynchronous I/O, memory-mapped I/O, DMA, and other methods (Zara et al., 2025). This switching abstraction allows the OS to better manage hardware access, shared memory, and I/O errors without allowing a faulty process to compromise the entire system.

If I were designing an OS, I would not remove the switching abstraction layer between user and kernel mode. However, I would use user mode drivers for lower-risk devices when possible, and keep kernel mode for memory access, DMA, permissions, and interrupts. I would also ensure that the I/O APIs are asynchronous by default, support safe ‘zero copy transfers’ to boost performance for large workloads, and implement stronger access permissions for sensitive devices such as cameras and microphones.

-Alex

References:

Colorado State University Global. (n.d.*). Lecture 7: Input output management* [Interactive lecture]. Canvas.

Microsoft. (2025, October 28). *User mode and kernel mode*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode

The Kernel Development Community. (n.d.). *Dynamic DMA mapping guide*. Linux kernel documentation. https://docs.kernel.org/core-api/dma-api-howto.html

Zara, D., Zafar, S., Hafeez, F., Azam, M., & Haider Ali, M. Z. (2025). Exploring advanced I/O techniques in modern operating systems. *Journal of Emerging Technology and Digital Transformation, 4*(1), 24–44.